EqlipZ Pay

A trust layer for payments made by people and by AI agents

Research notes and product plan, written for the Razorpay AI Buildathon 2026

What this is, in one line

EqlipZ Pay sits between a payment, whether it was started by a person or by an AI agent, and Razorpay's settlement rails. Instead of forcing a yes or no decision when the system is unsure, it holds the money for a short, bounded window, and it gets better over time by learning from how real disputes are resolved.

Part one lays out the research and the reasoning. Part two turns it into a working plan.

Contents

Part I. Research

1 Where this idea came from

I started with two separate lines of research. One looked at settlement risk inside Razorpay Route, the split payment rail that marketplaces use to pay vendors. The other looked at the new wave of AI agents that can now shop and pay on a person's behalf, using protocols like AP2 and UCP. They looked like two different hackathon ideas until I noticed they were pointing at the same gap: no payment system today, human facing or agent facing, is allowed to say "I am not sure yet" and still keep the transaction alive. It either approves the payment or it declines it. EqlipZ Pay is what happens when you take that gap seriously and build the missing middle option: a hold, backed by numbers, that resolves itself safely either way.

2 Why this matters right now

A few things happened in the last year that make this the right moment for it.

Razorpay itself has been pushing hard into AI native payments. Their 2026 product push, branded as the age of AI native payments, launched chat based checkout, autonomous purchasing inside chatbots, and a platform that is meant to think and act rather than just process. That surface area needs a safety net it does not currently have.

Razorpay also runs an official MCP server, the kind of tool interface that lets an AI agent, built on Claude, GPT, or anything else, call functions like capturing a payment or creating a transfer directly. That is a big deal, because it means an agent can already move real money through Razorpay with nothing standing in between the agent's decision and the actual transfer.

There is also a more direct reason. In August 2026, Razorpay launched Vulcan, a foundation model built with NVIDIA and AWS and trained on around four billion payments and roughly three trillion data points, reading close to three thousand signals per transaction. It handles routing, checkout personalisation, and fraud scoring at a scale nothing I could train myself would ever match. That is not competition for this project, it is the opposite of a problem. Vulcan is a very good predictor. It still has to hand its output to something that decides what to actually do with a transaction it is not fully sure about, and today that something is a plain threshold. EqlipZ Pay is built to sit right there, taking whatever score Vulcan produces, or a signal from Razorpay's Agent Studio agents, and turning it into a calibrated, three way decision with an action attached. I am not trying to out predict Razorpay's own model. I am building the part that comes after the prediction.

On the regulatory side, NPCI has been reported to be drafting a Unified Agent Protocol so that AI agents can transact over UPI. The RBI's own Payments Vision 2028 names AI driven fraud detection and agentic AI as priorities. Globally, Google's AP2, the Universal Commerce Protocol, and the x402 standard are all live and being adopted by large payment networks. Regulators in the UK and the US have separately flagged that nobody has really worked out what happens when an AI agent makes a bad purchase.

Put simply, the infrastructure that lets agents pay is moving faster than the infrastructure that keeps that safe. That gap is what I am building for.

3 The problems underneath the surface

Where	What actually goes wrong	What it costs
Marketplace payouts (Route)	Fraud is usually confirmed only after the settlement window has closed, by which point the vendor already has the money.	The marketplace eats the whole chargeback with no way to claw anything back.
Agent checkouts (AP2, UCP, MCP)	A poisoned product description can hijack an agent mid shop, and the cart it signs afterward still looks completely valid.	A new kind of chargeback that no existing fraud tool is built to catch, because the signature is real.

Where	What actually goes wrong	What it costs
Subscriptions (UPI AutoPay)	Retries happen on a fixed schedule instead of matching when the customer actually has money.	Recoverable revenue is lost purely because the timing was wrong.
Disputes	Small merchants gather evidence by hand against a hard deadline, with no sense of whether the case is even winnable.	Wasted effort, missed deadlines, and fees on cases that were never going to win.
Reconciliation	Webhooks get dropped or retried, and the merchant's records quietly drift from what Razorpay actually did.	Support tickets pile up and customers escalate complaints that were never really unresolved.

4 What already exists, and why it is not enough

Before settling on this direction, I looked hard for reasons someone might have already solved it.

Approach	Where it falls short	Closest example	What I do differently
LLM written dispute evidence	Prone to hallucination, and hallucination in a legal filing is not a small bug.	Chargehound, Midigator	I only generate evidence once a statistical check says the case is worth fighting.
Fraud scoring on device or behaviour	Blind to a server side agent that is technically behaving exactly as told.	Forter, Stripe Radar	I check whether the agent's cart still matches what the person actually asked for.
Fixed dunning schedules	Treats the symptom, not the reason the payment failed.	Chargebee, Churnkey	I time retries around the customer's real liquidity pattern.
Prompt injection filters	Can spot the attack but has no way to touch the money.	Various security tools	I wire detection straight into a Razorpay hold, so detection actually does something.
Reconciliation dashboards	Someone still has to notice the problem and act on it.	Truewind, Blueonion	The system acts on its own, through the same decision layer used for risk.

5 The idea in plain words

Core idea

A cryptographic signature tells you who authorised a payment. A fraud score tells you how risky it looks. Neither one tells you whether it is actually safe to release the money right now. That question only has three honest answers: release it, refuse it, or hold it a while. Razorpay Route is the only piece of infrastructure in Indian payments that lets you express a hold as a real, time bound, reversible state. So instead of building yet another classifier, I built the decision layer that is allowed to say hold, and I use the same layer whether the transaction came from a person or from an AI agent.

6 How this lines up with the AI Risk Manager track

The track brief asks for something specific: stop the merchant losing money to fraud, returns and chargebacks, and prove it with a working detector, verifier or auto responder for one class of loss, measured honestly with precision, recall, and false positive cost on a held out test set, and nothing in the system should be capable of offense, only defence.

That maps onto this project cleanly. The one class of loss I am measuring for the prototype is fraud on Route settlements, the same chargeback problem in the very first row of the problem table above. Every other piece, the semantic check on agent carts, the dunning timing, the dispute responder, sits on the same decision layer and reuses

the same calibration, but the number I report is precision and recall on that one class, on a held out split, with false positives converted into actual rupee cost through the EMV metric described later in this document. Nothing here scans for a way in, probes an endpoint, or takes any action other than detect, verify, hold, refuse, or respond to a dispute. It only ever acts on Razorpay's own settlement and dispute APIs, in the defensive direction.

Example direction	What a first pass usually looks like	Where this design goes further
Chargeback evidence responder	An LLM writes evidence for every dispute it sees	Evidence only gets generated once the conformal check says the case is actually worth fighting, so effort and arbitration fees are not wasted on cases that were never winnable
Fraud spike detector	A fixed threshold fires an alert when volume looks unusual	The calibration window tracks coverage drift directly, so a spike shows up as the accuracy guarantee slipping, not just as a raw count going up
Abuse ring sentinel	Rules flag repeated devices or IPs	The same graph aware conformal scoring that feeds the main engine also catches identities that look clean individually but sit in a suspicious neighbourhood of related transactions
Return risk scorer	A model scores the order at checkout and stops there	The score feeds into the same release, refuse, hold decision, so a risky order can be held rather than only flagged

7 Five things that make this different

These are the parts that do not exist in either of the original research threads on their own, and are the reason it should hold up under scrutiny.

7.1 One kernel for both kinds of buyers

Instead of building separate systems for human fraud and agent fraud, I built one decision layer with two ways of gathering evidence. For a human transaction, it wraps a conformal calibration layer around whatever risk score is already available, Razorpay's own Vulcan model where I can reach it, a locally trained fallback where I cannot, and turns that single number into a mathematically bounded confidence set instead of a guess. For an agent transaction it uses a second, read only model that checks whether the signed cart still matches what the person originally asked for. Both feed into the same calibrated decision layer, so the system has one consistent accuracy guarantee no matter who initiated the payment, and no matter which upstream model produced the first score.

7.2 Sitting inside the agent's own tool calls

Because Razorpay already exposes an MCP server, an agent can call functions like capture payment or create transfer directly. The same is true for agents built on Razorpay's own Agent Studio, launched earlier in 2026 for tasks like payment handling and revenue recovery. I sit as a proxy in front of that tool layer, wherever it comes from. Every call gets checked first. A safe call goes through untouched, an uncertain one gets rewritten to include a hold, and a clearly bad one never reaches Razorpay at all. This means I am governing the actual decision point, not just watching traffic go by, and it should keep working even as new agent protocols show up.

7.3 A trust record that travels with the vendor

When a held transaction is cleared, that outcome is recorded as a small, hashed reputation credential tied to the vendor or agent, not the raw transaction details. That credential can travel to the next marketplace using EqlipZ Pay, so a vendor who has already proven trustworthy is not treated like a stranger every single time. This also solves a real failure mode I noticed early: over holding a vendor's funds too often will eventually drive them away.

7.4 A system that learns from real outcomes

Every dispute that gets resolved through Razorpay's Disputes API, whether it is accepted or contested, won or lost, feeds back into the calibration on a rolling basis. That closes a loop that most competing tools leave open: a model trained once and left alone drifts. It keeps re earning its accuracy guarantee from real dispute outcomes, and the longer it runs on a merchant's data, the harder it becomes to copy.

7.5 Built with India's own agent payment standard in mind

NPCI has been reported to be working on a Unified Agent Protocol for agentic UPI payments. I built a thin adapter that maps AP2 style mandates onto the kind of fields that standard is likely to expect. I am not betting the whole product on the final spec, just making sure I am not starting from zero when it lands.

8 Where the ideas underneath this come from

None of the individual techniques here are new on their own. What is new is putting them together in one payment kernel.

- Conformal prediction and Mondrian conformal prediction give distribution free accuracy guarantees even with heavily imbalanced fraud data.
- Selective conformal risk control extends that into a framework for deciding when to act and when to abstain, which is the mathematical backbone of the release, refuse, hold router.
- Temporal graph conformal methods help catch fraud that hides inside an otherwise normal looking transaction history.
- Process mining, using something like the PM4Py library, rebuilds the expected shape of a payment's lifecycle so dropped webhooks can be caught automatically.
- Zero knowledge style attestations are what let the trust passport prove a history without leaking the transactions behind it.

9 How this compares

Capability	**EqlipZ Pay**	**Radar, Forter**	**Charge-hound**	**Churn-key**	**Prompt filters**
Statistical accuracy guarantee	Yes	No	No	No	No
Checks agent intent, not just signature	Yes	No	No	No	Partial
Can actually hold the money, not just flag it	Yes	No	No	No	No
Sits inside agent tool calls	Yes	No	No	No	No
Learns from real dispute outcomes	Yes	Partial	No	No	No
Trust that travels across merchants	Yes	No	No	No	No
Built for India's UPI agent direction	Yes	No	No	No	No

10 Questions I expect to get asked

Anticipating the pushback

Isn't this just Stripe Radar or Forter with extra steps? Those tools score and block at the gateway. I act after authorisation, at settlement, which is the only point where a reversible hold is even possible.
AP2 already handles authorisation, so what's left to solve? AP2 proves the signature is valid. It says nothing about whether the agent was tricked into signing something it should not have.
Why not just block anything uncertain? Blocking kills the sale outright. Holding keeps the sale alive while capping how much the merchant can lose if it turns out to be fraud.
What if the model gets it wrong? That is exactly what the statistical guarantee is for. It bounds the error rate mathematically instead of hoping the model is right.
Doesn't Razorpay's own Vulcan model already do fraud detection? Vulcan predicts. It does not, on its own, decide to hold a transfer for 48 hours or route a case straight to the disputes flow. EqlipZ Pay is the calibration and action layer that turns any prediction, Vulcan's included, into a bounded decision with a real API call behind it.

I was not able to find any existing research or product that combines conformal risk control, agent intent checking, MCP level interception, and programmable Route escrow the way this does.

Part II. Product plan

11 What I am building

EqlipZ Pay is risk and trust middleware for Razorpay merchants and for the agent facing checkouts that are starting to appear on top of Razorpay. The goal is for it to become the default safety layer sitting underneath every AI native payment surface Razorpay ships, for human buyers and autonomous agents alike.

12 Goals, and what I am deliberately not doing

- Replace the simple approve or decline decision with a calibrated release, refuse, or hold decision that carries a real accuracy guarantee.
- Govern agent initiated Razorpay tool calls before they execute, not after.
- Close the loop using actual Disputes API outcomes instead of a model trained once and forgotten.
- I am not building a new fraud dataset from scratch. The prototype uses BankSim plus synthetic AP2 and UCP payloads. A production version would learn from the merchant's own history.
- I am not trying to replace Razorpay's checkout, its own fraud tooling, or KYC. This sits on top, as a layer, not a competing gateway.

13 Who this is for

Person	What they need	How EqlipZ Pay helps
Marketplace risk lead	Stop absorbing vendor chargebacks after the money has already settled.	Holds ambiguous Route transfers automatically for up to 48 hours.
Platform engineer	Let agents check out through MCP without opening a new fraud hole.	Every transfer and capture call from an agent is checked before it runs.
SMB finance owner	Stop guessing which disputes are worth fighting.	Weak cases are conceded automatically, strong ones get evidence generated for them.
Subscription merchant	Stop losing revenue to badly timed retries.	Retry timing is folded into the same decision layer, based on real liquidity signals.

14 System architecture

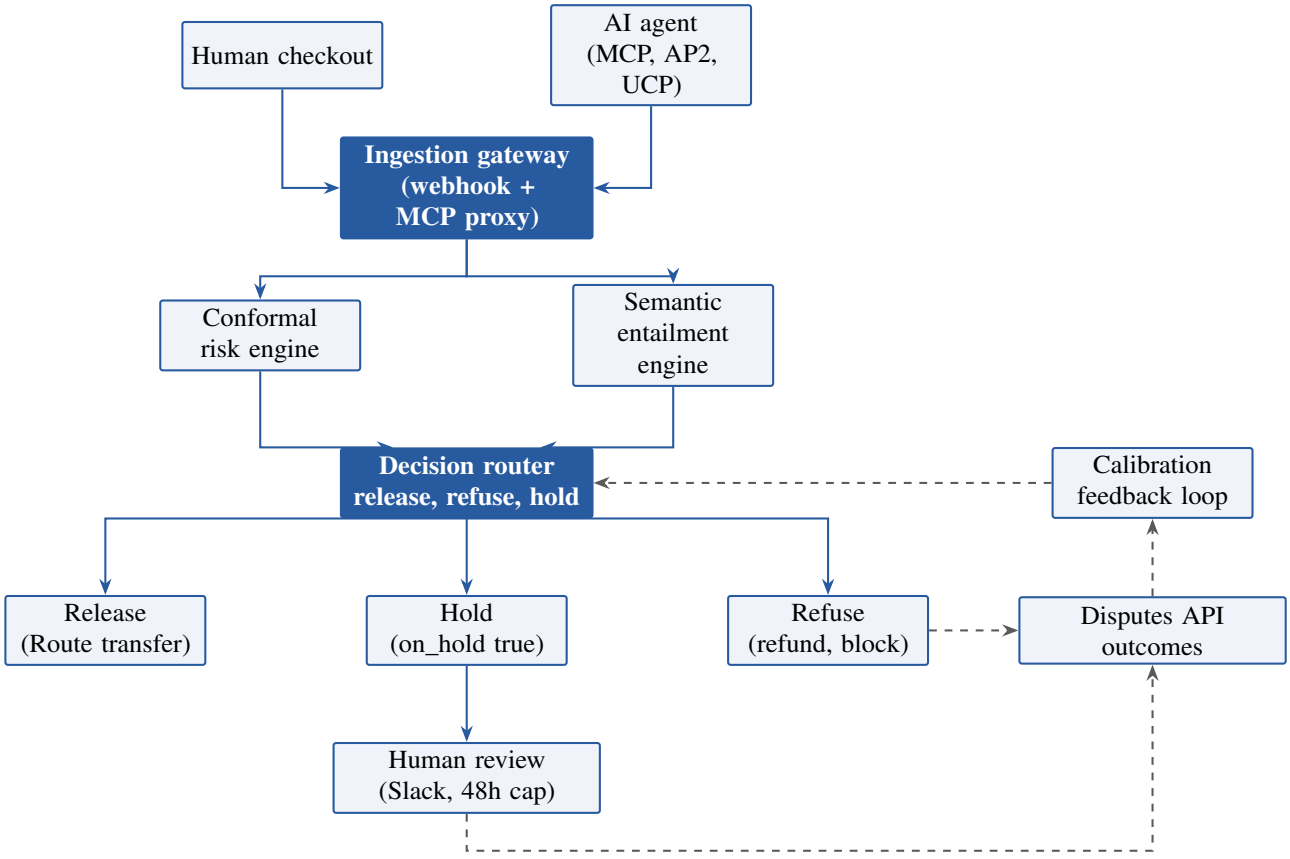


Figure 1: EqlipZ Pay system architecture. Solid arrows show the live decision path. Dashed arrows show the feedback loop that recalibrates the router using real Disputes API outcomes.

15 How data moves through the system

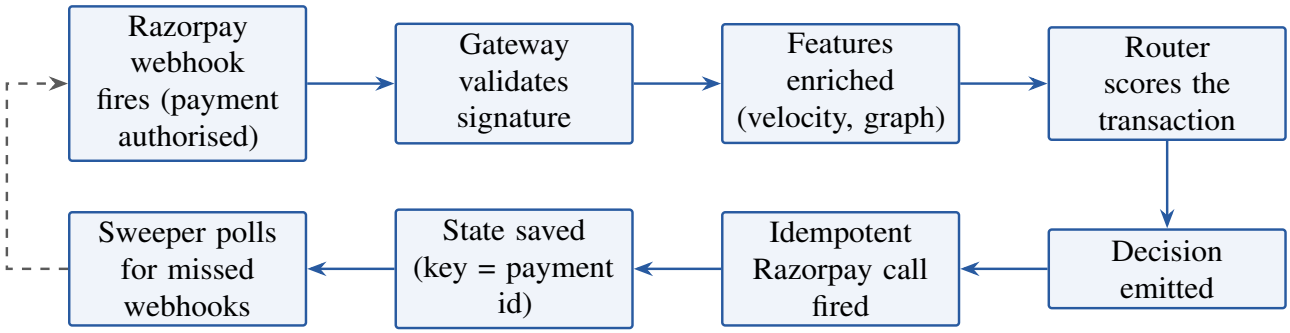


Figure 2: Data flow from webhook to executed action. A sweeper running every 15 minutes covers any webhook that Razorpay's own retry window does not resolve.

16 How the escrow decision resolves

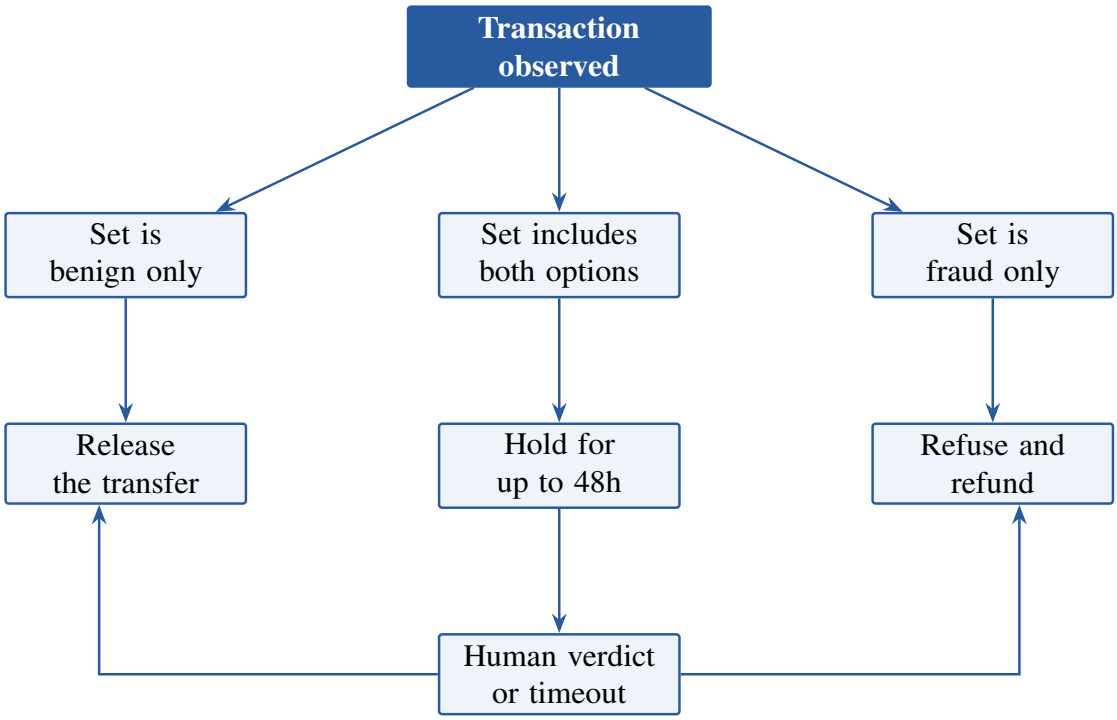


Figure 3: If a hold is never resolved by a person, it defaults safely to release once the 48 hour cap is reached, so a vendor's funds are never withheld indefinitely.

17 What each module does

Module	Job	Talks to
Ingestion gateway	Verifies webhook signatures and proxies MCP tool calls	Razorpay webhooks, MCP server
Conformal risk engine	Calibrates a bounded accuracy guarantee on top of Vulcan's score, or a local fallback model in test mode	Vulcan risk score, MAPIE
Semantic entailment engine	Checks an agent's cart against its original intent	Read only LLM
Decision router	Combines both engines under one shared guarantee	Internal only
Action executor	Fires idempotent Razorpay calls	Transfers, refunds, disputes
Calibration feedback loop	Recomputes thresholds from dispute outcomes	Disputes API, rolling 24h
Trust passport service	Issues and checks portable, hashed risk credentials	Internal, cross merchant
Sweeper	Polls for webhooks that never arrived	GET payments

18 APIs and tool calls involved

- Creating a Route transfer, with the hold fields set by the router when needed.
- Modifying a settlement hold, to release it early or extend it.
- Refunding a payment, for the clearly fraudulent path.
- Contesting or accepting a dispute, which also feeds the calibration loop.
- Polling payments directly, for the sweeper's reconciliation pass.

- Proxying the MCP tools that Razorpay's own MCP server exposes, such as capturing a payment or creating an instant settlement.
- Every write carries the payment ID as an idempotency key, so a retried webhook never causes a duplicate transfer.

19 Data model

Entity	Key fields	Notes
Transaction	payment id, amount, source, status	An immutable event log
RiskDecision	transaction id, prediction set, confidence, action	One record per transaction
EscrowHold	transfer id, hold until, verdict, resolved by	Lives for at most 48 hours
DisputeOutcome	dispute id, phase, result, evidence reference	Feeds the calibration set
TrustPassport	entity id, hold count, benign count, credential hash	Portable, hashed only

20 What the system must do

1. Take in Razorpay webhooks and MCP tool calls through one shared interface.
2. Produce a prediction set, not a single score, for every transaction.
3. Map a benign only set to release, a fraud only set to refuse, and a mixed set to hold.
4. Auto release any hold that has not been resolved within 48 hours.
5. Make every write to Razorpay idempotent using the payment ID.
6. Feed every resolved dispute back into the calibration set within 24 hours.
7. Keep trust passport credentials portable across merchants without exposing raw transaction data.

21 Non functional requirements

Requirement	Target
Accuracy guarantee	At least 90 percent marginal coverage on the calibration set
Decision speed	Under 500 milliseconds for a release or refuse decision
Escrow limit	Hard cap of 48 hours, never an indefinite hold
Idempotency	No duplicate transfers even under a webhook retry storm
Resilience	Survives Razorpay's own 24 hour webhook retry window
Privacy	The trust passport stores only hashed credentials, never raw personal data

22 Dataset strategy

This section locks down where the training and evaluation data actually comes from, split by what each source is trusted to prove.

Design decision

Rather than a single dataset, the project draws on three independent real or research grade sources for fraud, plus one benchmark built in house for the one problem no public dataset covers: whether an AI agent's final cart still matches what the person actually asked for.

Dataset	Type	Use in EqlipZ	Priority
IEEE-CIS Fraud Detection	Real world e-commerce transaction data	Main fraud model	Highest
ULB Credit Card Fraud	Real world card transactions	Independent validation	High
Fraud Detection Handbook simulator	Synthetic, research designed	Stress testing, controlled scenarios	Medium
Agent-intent benchmark, built in house	Synthetic, controlled	AI agent intent mismatch detection	Highest

22.1 IEEE-CIS: the primary dataset

This is the dataset the main fraud model is built around. It was provided by Vesta, a real payment fraud prevention company, for the IEEE-CIS competition, and it contains real world e-commerce transaction data spanning transaction, product, card, address, email, device and identity related features. It runs to 871 columns across the transaction and identity files, roughly 1.35 GB in the original Kaggle distribution. Source: IEEE-CIS Fraud Detection, Kaggle. The pipeline for this source is: real world transaction, fraud model, probability of fraud, conformal calibration. This becomes the main evidence behind the Track 02 claim.

22.2 ULB Credit Card Fraud: independent validation

This dataset contains 284,807 transactions, including 492 confirmed frauds, a fraud rate of only 0.172 percent, from a collaboration between Worldline and the Machine Learning Group at ULB. Its main limitation is that most features are anonymised PCA components, so it is less useful for explainability, but it answers a different question well: does the risk methodology generalise beyond the primary dataset. Source: ULB Credit Card Fraud Detection, Kaggle.

22.3 Fraud Detection Handbook simulator: controlled stress testing

This one is not real data and is clearly labelled as synthetic throughout. It is useful precisely because it intentionally models the hard parts of fraud detection rather than the easy parts:

- severe class imbalance
- a mix of categorical and numerical variables
- customer and merchant behaviour patterns
- temporal patterns in spending
- fraud behaviour that changes over time
- multiple distinct fraud scenarios, not just one

The authors describe it explicitly as an approximation of reality built for reproducible fraud detection research. Source: Fraud Detection Handbook, transaction simulator and GitHub repository. This source is used for stress tests, not for the headline result.

22.4 Agent-intent dataset: built in house

This is the one part of the problem where no good public real dataset exists, because it is not really a fraud question. IEEE-CIS can answer "is this transaction likely fraudulent". It cannot answer "did the AI agent buy what the human actually intended". So a separate benchmark is generated, following the pipeline: user intent, agent plan, final cart, intent alignment label.

Example: aligned

Intent: "Buy the cheapest 16GB laptop under Rs.80,000."
Agent action: Rs.74,999 Lenovo laptop, 16GB.
Label: ALIGNED.

Example: mismatch

Intent: "Buy the cheapest 16GB laptop under Rs.80,000."
Agent action: Rs.1,47,999 MacBook.
Label: MISMATCH.

The benchmark generates a mix of legitimate, ambiguous, and malicious or mismatched scenarios, and is explicitly documented as a controlled benchmark rather than real world fraud data. This distinction matters for research integrity: claiming synthetic intent-alignment scenarios as real fraud evidence would undermine the credibility of the whole submission, so the separation is kept visible everywhere the numbers are reported, not just in a footnote.

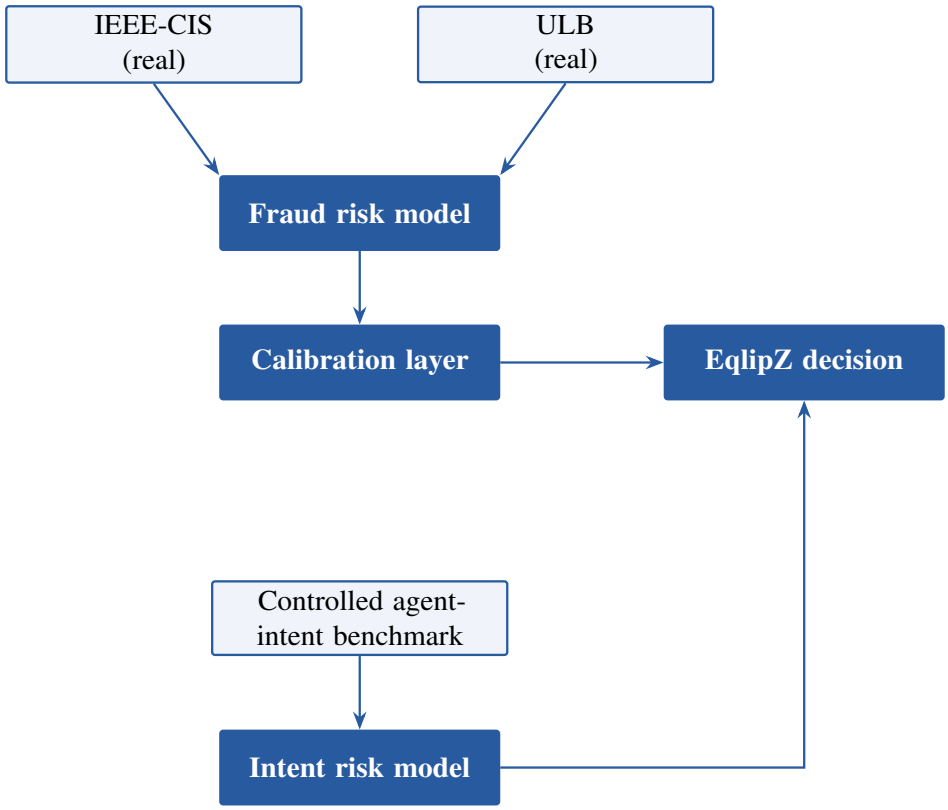


Figure 4: Data architecture: real world fraud data and the controlled agent-intent benchmark feed two separate models that converge in the same EqlipZ decision layer. The Fraud Detection Handbook simulator sits underneath both as a stress-testing source, not shown here to keep the diagram readable.

Caveat to state plainly in the README and the pitch

None of the public real world datasets contain actual Razorpay transactions or actual AI agent transactions. Every number reported from IEEE-CIS or ULB is a statement about fraud detection methodology in general, not a statement about Razorpay's own traffic. The Fraud Detection Handbook authors themselves note how scarce publicly shareable real payment card data is, for exactly this confidentiality reason. Every result in the submission is therefore labelled by which source it came from, real world data or the controlled agent-intent benchmark, and that separation is what makes the project more credible, not less.

23 Layered product architecture

The build is locked around a single description: EqlipZ is an API-first AI risk infrastructure layer, with an Agent Gateway in front of it, a web control plane on top of it, and a Razorpay test-mode integration underneath it. It is not a choice between an API, a web app, and a model. It is all of them, organised as layers with one core.

Positioning, in one line

EqlipZ is an API-first AI risk infrastructure layer. It sits between an AI agent and payment execution, with a web control plane for monitoring, explanations, and auditability.

23.1 Layer 1: the EqlipZ Risk API (core)

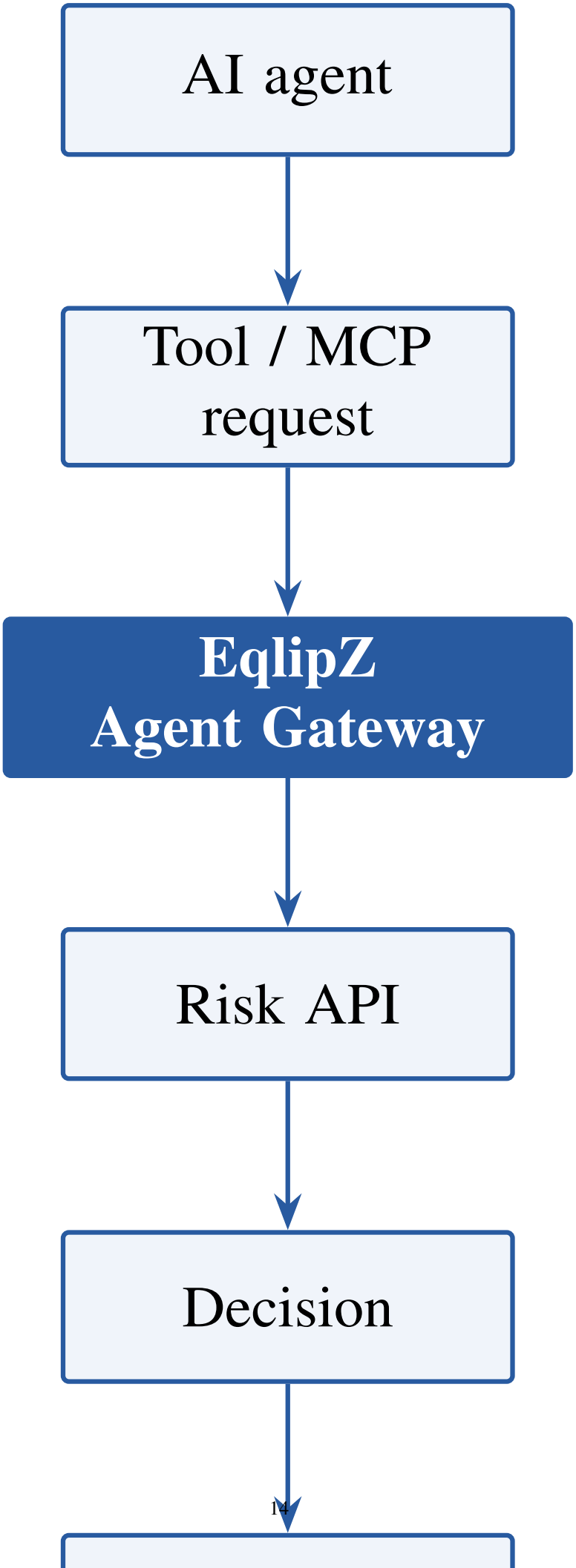
This is the actual product. Every other layer is a client of it.

```
POST /v1/risk/evaluate

Request body:
{
  "transaction": { ... },
  "user_intent": "...",
  "cart": [ ... ],
  "agent_context": { ... }
}

Response body:
{
  "decision": "HOLD",
  "risk_score": 0.71,
  "intent_alignment": 0.43,
  "prediction_set": ["BENIGN", "FRAUD"],
  "reason_codes": ["intent_mismatch", "unusual_transaction"],
  "audit_id": "..."
}
```

This is what makes EqlipZ look like something a real payment platform could actually integrate against, rather than a hackathon demo with no clean edge.

23.2 Layer 2: the Agent Gateway

This is the layer that turns a generic fraud API into an AI agent risk manager. An AI agent's tool or MCP request never reaches Razorpay directly; it is intercepted, scored by the Risk API, and only then allowed, rewritten to include a hold, or blocked outright.


AI agent
Tool / MCP request
EqlipZ Agent Gateway
Risk API
Decision

23.3 Layer 3: the web control plane

This is what judges and merchant risk teams actually look at. It is a dashboard over the same audit log the API and gateway already write to, not a separate source of truth.

Risk overview	
Transactions	12,481
Fraud detected	438
Held	127
Prevented loss	Rs.4.82L
False-positive cost	Rs.31K

Transaction	**Amount**	**Risk / alignment**	**Prediction set**	**Decision**
EP-10291	Rs.74,999	Risk 18%, alignment 96%	{BENIGN}	RELEASE
EP-10302	Rs.1,47,999	Risk 84%, alignment 31%	{FRAUD}	REFUSE
EP-10311	Uncertain	—	{BENIGN, FRAUD}	HOLD, expires 47:12:33

This view is where the conformal prediction set becomes visually understandable to someone who has never heard of conformal prediction: they just see which transactions the system refuses to guess on.

23.4 Layer 4: Razorpay integration

The demo runs entirely on Razorpay Test Mode, never real money. The path is: AI agent, EqlipZ, risk decision, Razorpay Test Mode, webhook back to EqlipZ, audit trail. This is the actual payment infrastructure connection the buildathon requires, without touching production funds.

23.5 Layer 5: evaluation and research, hidden behind the product

The repository must be able to produce an honest evaluation report, run from real experiments rather than written by hand.

```
MODEL EVALUATION

Dataset: IEEE-CIS
Transactions: XXXXX

Precision: XX.X%
Recall: XX.X%
F1: XX.X%
AUCPR: XX.X%

False-positive cost: Rs. XX,XXX
False-negative cost: Rs. XX,XXX

Conformal coverage: XX.X%

GENERALIZATION

Dataset Precision Recall
IEEE-CIS XX.X% XX.X%
ULB XX.X% XX.X%
Handbook Simulator XX.X% XX.X%
```

These numbers are not to be manufactured ahead of time. They are only reported once the experiments that produce them have actually been run.

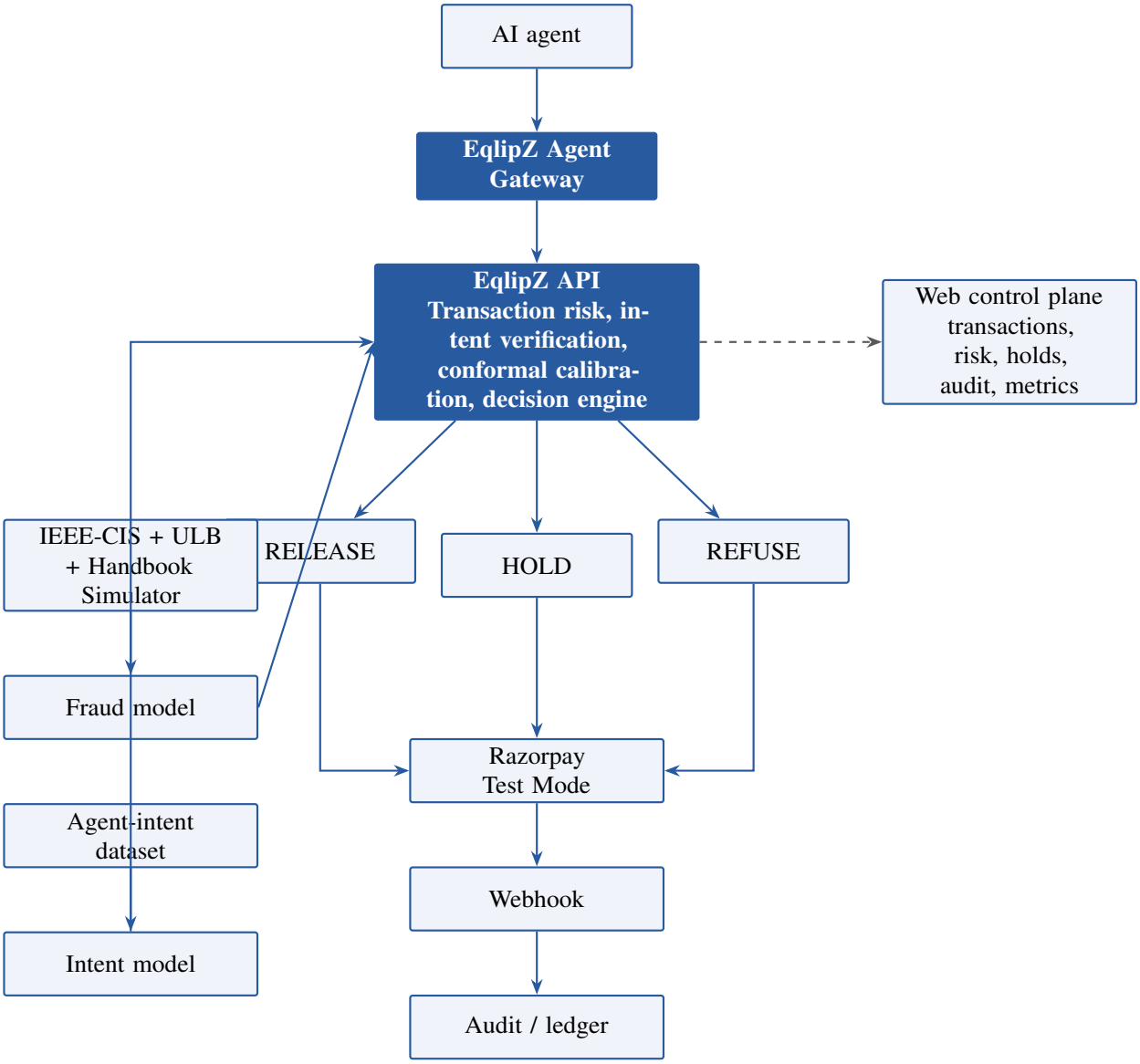


Figure 6: Final layered architecture. The agent gateway, risk API, decision fan-out, Razorpay test mode path, audit trail, web control plane, and the two underlying data pipelines, fraud and agent intent, as one system.

23.6 What this is, when asked directly

If a judge asks: is EqlipZ an API, a web app, or an AI model?

The answer is that it is an API-first AI risk infrastructure layer. It sits between an AI agent and payment execution, with a web control plane for monitoring, explanations, and auditability. Core: API. AI: fraud, intent, and conformal risk engine. Agent interface: MCP and gateway. Payment: Razorpay Test Mode. Frontend: web control plane. Data: real fraud datasets plus a controlled agent benchmark. Output: release, hold, or refuse. Evidence: precision, recall, monetary cost, calibration, and generalisation.

This is deliberately not a fraud prediction dashboard, not a chatbot wrapped around Razorpay, and not a simulated payment app with no real backing. It is a working AI risk infrastructure layer that sits in the payment path of an AI agent, learns from real world fraud data, verifies agent intent, quantifies uncertainty, and gates payment execution through release, hold, and refuse decisions.

24 How I will know it works

- Marginal coverage of at least 90 percent on a held out mix of BankSim and synthetic AP2 data.
- Expected monetary value: revenue captured plus false positive revenue saved, minus chargeback losses from anything that slipped through, compared against a simple threshold based baseline.

- Median time from a hold being placed to a human verdict.
- The share of injected, malicious agent carts that actually get routed to hold or refuse.

25 How the project is organised

```
eqlipz-pay/
|-- ingestion/
|   |-- webhook_listener.py
|   +-- mcp_proxy.py
|-- risk_kernel/
|   |-- conformal_engine.py
|   |-- semantic_engine.py
|   +-- decision_router.py
|-- actions/
|   |-- route_transfer.py
|   |-- disputes_client.py
|   +-- refund_client.py
|-- flywheel/
|   |-- calibration_job.py
|   +-- trust_passport.py
|-- sweeper/
|   +-- reconcile_cron.py
|-- config/
|   |-- razorpay_keys.env
|   +-- thresholds.yaml
|-- tests/
+-- docs/
    |-- research.pdf
    +-- prd.pdf
```

Figure 7: Repository layout for the prototype.

26 Risks I am watching

Risk	What happens	How I handle it
A hold sits too long	The vendor gets frustrated and may leave	Hard 48 hour cap that auto releases
A webhook never arrives	A risk decision gets missed entirely	Sweeper polls every 15 minutes and replays safely
The model drifts over time	The accuracy guarantee stops holding	Rolling 24 hour recalibration from real disputes
Regulatory concern about holding funds	Possible compliance push-back	I rely on the hold feature Razorpay already documents for this exact purpose
MCP interception adds delay	Agent checkout feels slower	Scoring happens asynchronously, within a half second budget

27 How this could become a real product

I do not see this as competing with Razorpay's checkout. It is the layer that lets Razorpay's own AI native features scale without taking on more risk than they can see. The path I would follow starts with a pilot, something like a

Route risk guard toggle for a marketplace partner, priced as a small fee on the volume it protects. From there it could grow into a standalone risk service that any Razorpay merchant exposing an MCP or AP2 surface can plug into.

References

Razorpay documentation and product pages:

- Modify settlement hold for transfers. https://razorpay.com/docs/api/payments/route/modify-settlement-hold/
- Create a direct transfer. https://razorpay.com/docs/api/payments/route/direct-transfers/
- Linked accounts and settlement schedules. https://razorpay.com/docs/payments/route/linked-account/
- Route product overview. https://razorpay.com/route/
- Webhooks, retries, and delivery behaviour. https://razorpay.com/docs/webhooks/
- Subscription payment retries and dunning. https://razorpay.com/docs/payments/subscriptions/payment-retries/
- Accept a dispute. https://razorpay.com/docs/api/disputes/accept/
- Contest a dispute. https://razorpay.com/docs/api/disputes/contest/
- TPAP Pro complaints flow. https://razorpay.com/docs/api/payments/tpap-pro/complaints-flow/
- Razorpay escrow accounts. https://razorpay.com/x/escrow-accounts/
- Razorpay Sprint 2026, the AI native payments launch. https://razorpay.com/sprint/26
- Official Razorpay MCP server. https://github.com/razorpay/razorpay-mcp-server

News coverage used for the market context in this document:

- Razorpay builds proprietary AI model Vulcan to boost payment success rates, Business Standard, August 2026. https://www.business-standard.com/industry/news/razorpay-builds-proprietary-ai-model-vulcan-to-boost-payment-success-rates-126081801078_1.html
- Razorpay launches Vulcan, an AI foundation model for payments, but what does it train on, Medianama, August 2026. https://www.medianama.com/2026/08/223-razorpay-vulcan-ai-foundation-model-payments/
- India may allow agentic AI led UPI transactions under new NPCI protocol, Business Standard, July 2026. https://www.business-standard.com/finance/news/india-may-allow-agentic-ai-led-upi-transactions-under-new-npci-protocol-126070801343_1.html
- Reserve Bank of India's Payments Vision 2028, Global Government Fintech, April 2026. https://www.globalgovernmentfinance.com/rbi-payments-vision-2028/
- Is 2026 the year of agentic payments, Fenwick and West, April 2026. https://www.fenwick.com/insights/publications/is-2026-the-year-of-agentic-payments
- Agentic commerce in 2026, AP2, x402, and AI payments. https://www.bitontree.com/agentic-commerce-ai-agents-payments-ap2-x402
- How agentic AI will reshape payments, IMF Notes, 2026. https://www.elibrary.imf.org/view/journals/068/2026/004/article-A001-en.xml

Research this design draws on:

- Temporal graph prototype conditioned conformal prediction for fraud detection. https://arxiv.org/pdf/2608.15768
- Elements of conformal prediction for statisticians. https://arxiv.org/pdf/2603.23923
- Conformal prediction and trustworthy AI. https://arxiv.org/html/2508.06885v1
- Class conditional conformal prediction for reliable anomaly detection under extreme class imbalance, MDPI. https://www.mdpi.com/2504-4990/8/7/190

- Selective conformal risk control. https://arxiv.org/pdf/2512.12844
- Process discovery using graph neural networks. https://arxiv.org/pdf/2109.05835
- A decision centred reference architecture for trustworthy agentic commerce. https://arxiv.org/html/2607.18347v1

Datasets referenced in the dataset strategy section:

- IEEE-CIS Fraud Detection, Kaggle. https://www.kaggle.com/c/ieee-fraud-detection
- Credit Card Fraud Detection (ULB / Worldline), Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- Fraud Detection Handbook, transaction simulator. https://fraud-detection-handbook.github.io/fraud-detection-handbook/
- Fraud Detection Handbook, GitHub repository. https://github.com/Fraud-Detection-Handbook/fraud-detection-handbook

A short glossary

- **AP2**: Google's Agent Payments Protocol, which uses signed mandates to authorise a purchase.
- **UCP**: the Universal Commerce Protocol, a shared way for agents to read merchant catalogs.
- **UAP**: NPCI's proposed protocol for letting agents make UPI payments.
- **MCP**: the Model Context Protocol, the standard agents use to call external tools.
- **SCRC**: selective conformal risk control, the statistical basis for the decision router.
- **EMV**: expected monetary value, the business metric used here instead of raw model accuracy.